

넓은 필드에서 누구를 살려두고, 얼마나 자주 돌릴 것인가

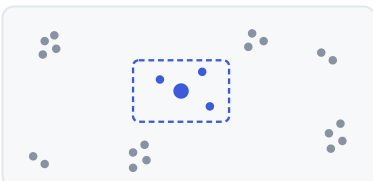
현재 Sector 방식을 출발점으로, 필드 엔티티(몬스터·NPC)를 관리하는 방식들을 그림으로 나란히 놓는다. 결론을 먼저 정하지 않고, 각 방식이 무엇을 풀고 무엇을 남기는지를 본다.

0. 질문부터 쪼갬다 — 세 장면은 서로 다른 문제다

"필드 몬스터 관리"라고 한 단어로 부르지만, 장면마다 비용을 만드는 변수가 다르다.

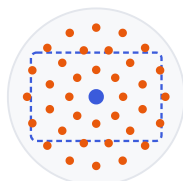
① 이 트랙이 다루는 세 장면

① 연속된 일반 필드 (로아 필드형)



넓다 · 수백 마리가 무리로 흩어져 있다
화면(점선)에 보이는 건 그중 일부분
핵심 변수 = 넓이 (공간)

② 좁은 아레나 대량 (카오스 던전형)



좁다 · 한 화면에 수십~수백이 몰려든다
거의 전부 화면 안 — 공간으로 거를 게 없다
핵심 변수 = 밀도

③ 오픈월드 · 반오픈월드



매우 넓다 · 지역 단위로 로드/언로드
멀리 있는 세계는 '데이터'로만 존재
핵심 변수 = 스트리밍 (로드)

①은 "어디를 켤까", ②는 "켜진 것을 얼마나 싸게 돌릴까", ③은 "무엇을 메모리에 둘까"가 문제다. 한 방식으로 셋을 다 풀 수 없다.

② 층으로 보면 — 층마다 '반경'이 다르다 이 문서의 좌표계

① 틱 (품질)
얼마나 자주 생각하나

매 프레임

가끔

② 존재 (GameObject)
씬에 오브젝트가 있나

켜져 있음

③ 로드 (에셋·지형)
메모리에 올라와 있나

에셋 로드됨 (오브젝트는 없을 수 있음)

④ 레코드 (데이터)
상태·타이머가 기록돼 있나

맵 전체 — 항상 (스�폰너 상태 · 리스폰 시각 · 처치 기록)

플레이어

≈화면

≈화면×2

훨씬 멀리

맵 끝

위로 갈수록 비싸고 반경이 좁다. 이 문서의 A~D는 결국 "②존재 층의 반경을 무엇으로 정하느냐"의 차이이고, ①·③·④는 그와 별개로 겹쳐 쓰는 층이다. (거리 수치는 개념도)

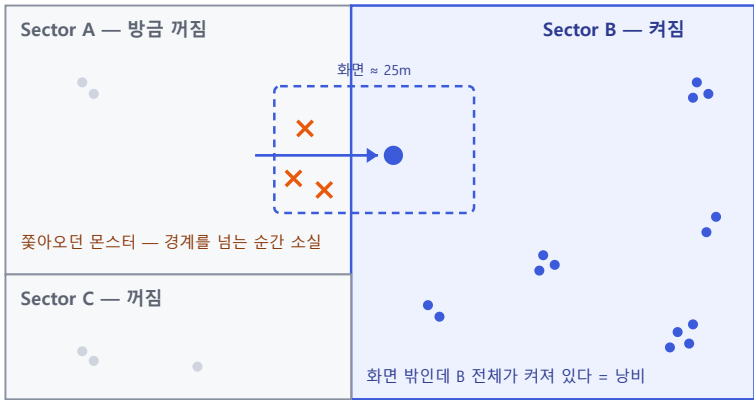
NPC도 같은 틀에 들어간다

NPC는 대개 움직이지 않고 전투도 안 하므로 ②존재 층만 관리하면 되고 ①틱 비용은 거의 0이다. 이 문서는 몬스터로 설명하지만, "정적인 것은 공간으로 판정한다"는 규칙은 NPC·채집물·상호작용 오브젝트에 그대로 적용된다.

1. 출발점 — 현재 Sector 방식과 경계 문제

트리거 볼륨(Sector)에 들어가면 그 구역 스포너를 전부 켜고, 이전 구역은 통째로 끈다. `SectorManager.OnPlayerEnterSector`

③ 플레이어가 A→B로 넘어간 직후 (현재 규모: Sector 5 · 스포너 23 · 몬스터 363)

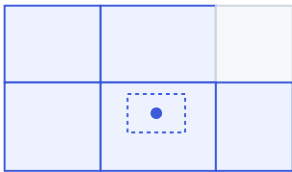


- 1 · 경계 소실
구역을 넘는 순간 이전 구역이 통째로 꺼진다
싸우던 몬스터가 눈앞에서 사라진다
- 2 · 판정 단위 > 화면
Sector ≈ 100m vs 화면 ≈ 25m
안 보이는 곳을 대량으로 켜 둔다
- 3 · 저장 노가다
구역을 사람이 그리고 ID를 매긴다
맵이 바뀌면 구역도 다시 손봐야 한다
- 4 · 진입 스파이크
진입 프레임에 구역 전체 스폰이 몰린다
(가장 큰 구역 = 한 번에 119마리)

1번이 눈에 보이는 버그지만, 2·3번이 구조의 한계다. 1번만 고치면 2·3·4번은 그대로 남는다.

④ 떠오르는 두 가지 패치 — 둘 다 '증상'만 막는다

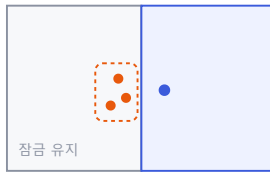
패치 ① 인접 Sector도 같이 켜기



화면(점선)은 그대로인데 켜진 영역만 커진다

- ✓ 경계 소실이 거의 사라짐
- X 켜는 면적 x2~3 (낭비 증가)
- X '누가 이웃인가'를 또 손으로 관리
- X 이웃 목록이 조용히 틀려도 모름

패치 ② 전투 중인 몬스터는 끄지 않기



리쉬(집으로 복귀)는 어떤 방식을 골라도 필요 — 이 작업은 버려지지 않는다

- ✓ 경계 소실 해결
- X 낭비·스파이크·노가다 그대로
- X 경계에서 계속 싸우면
두 구역이 계속 켜져 있음
- X 잠금 해제 누락 = 조용한 누수

원인은 하나

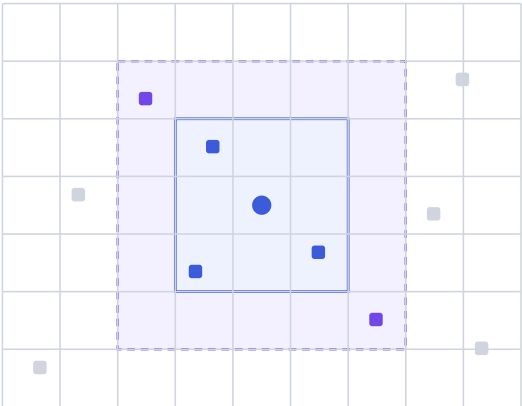
판정 단위가 사람이 그린 구역이라는 것. 구역이 화면보다 크고, 경계가 임의의 위치에 있고, 손으로 유지해야 한다. 그래서 다음 방식들은 모두 판정이 기하학(격자·거리)에서 자동으로 나오게 만드는 방향이다.

2. 방식 A — 맵 전체를 균일한 청크(격자)로 나눈다

좌표를 셀 크기 S로 나누면 셀 번호가 나온다. 구역을 그릴 필요가 없다. 플레이어 셀 + 주변 셀만 켜다.

⑤ 플레이어 주변 3×3 활성화, 5×5까지는 유지 (히스테리시스)

■ = 스포너 (정적)

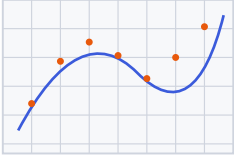


■ 활성화 3×3 — 여기 들어오면 켜 ■ 유지 5×5 — 커진 건 여기까진 안 끄

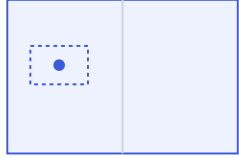
셀 번호 = ($\lfloor x / S \rfloor$, $\lfloor z / S \rfloor$)

위치만 알면 O(1)로 셀을 안다. 셀 → 스포너 목록은 해시맵 한 번.
맵을 바꿔도 다시 그릴 게 없다 — 로드 시 스포너 위치로 자동 등록.

셀 크기 S의 딜레마



S가 작으면: 경계를 자주 넘는다
켜고 끄기가 잦다 (churn)



S가 크면: 셀 하나가 화면보다 크다
= Sector와 같은 낭비로 돌아감

정사각형 효과

활성 영역이 네모라서 대각선 방향은 1.41배 멀리까지 켜진다.
화면 자체가 네모인 쿼티뷰에선 오히려 잘 맞을 수도 있다

경계 문제는 "주변 셀도 켜다(3×3)"로 사라지고, 경계선 왕복 깜빡임은 "켜는 범위 < 끄는 범위"(3×3 / 5×5)로 막는다.

풀어주는 것

- 저작 비용 0 — 구역을 그리지 않는다
- 셀 조회 O(1). 맵이 커져도 판정 비용이 그대로
- 셀은 예셋 로드 단위·저장 단위로도 쓸 수 있다 (리소스 시스템과 연결점)
- 같은 격자를 몬스터 이웃 탐색·광역기 판정에 재사용 가능

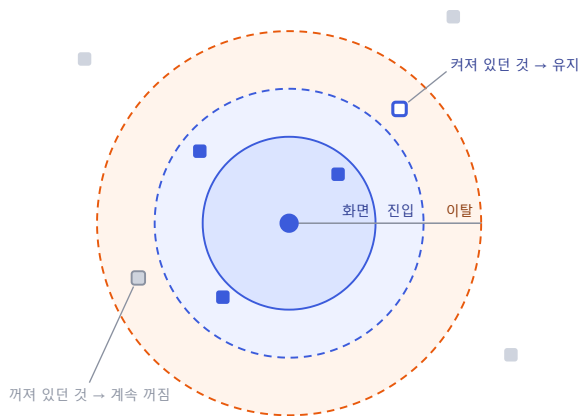
남기는 것

- 셀 크기 S를 골라야 한다 — 너무 작으면 churn, 크면 낭비
- 활성 영역이 네모 → 방향마다 켜지는 거리가 다르다
- 셀 경계를 넘을 때 3~5개 셀이 한꺼번에 바뀐다 (작은 스파이크)
- 밀도가 고르지 않은 맵에선 셀마다 비용 편차가 크다

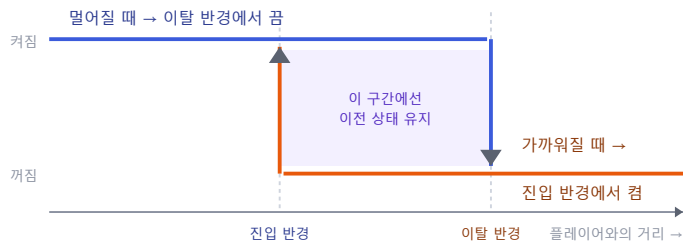
3. 방식 B — 플레이어와의 거리로만 판단한다

스포너마다 플레이어와의 거리를 재서 가까우면 켜고 멀면 끈다. 구역도 격자도 없다.

⑥ 거리 밴드와 히스테리시스



왜 반경이 두 개인가 — 히스테리시스



Unity 기본 제공 — CullingGroup

```
SetBoundingSpheres(스포너 위치들) // 로드 시 1회  
SetBoundingDistances(진입, 이탈)  
SetDistanceReferencePoint(플레이어) // 카메라 X  
onStateChanged += 밴드가 바뀐 스포너만 통보
```

반경이 하나면 경계선 위에서 왔다 갔다 할 때 매 프레임 켜졌다 꺼진다. 켜는 선과 끄는 선을 떨어뜨리면 그 사이에선 이전 상태가 유지된다.

풀어주는 것

- 경계가 존재하지 않는다 — 고친 게 아니라 없앤 것
- 고정 각도 쿼터뷰에선 "플레이어 중심 반경 = 화면"이라 화면에 딱 맞게 켤 수 있다
- 저작 비용 0, 격자 크기 고민도 없다
- 한 번에 켜지는 건 반경에 막 들어온 몇 개뿐 → 진입 스파이크가 자연히 분산

남기는 것

- 전수 검사 — 매번 모든 스포너와 거리를 쟀다 (N이 커지면 비례)
- 팝인 — 반경에 들어오는 순간 없던 게 생긴다 (반경을 화면보다 넉넉히)
- 자유 카메라 3D에선 "거리 = 시야"가 깨진다 (멀리 보이는 산 위 몬스터)
- 밀집엔 무력 — 다 가까이 있으면 거를 게 없다

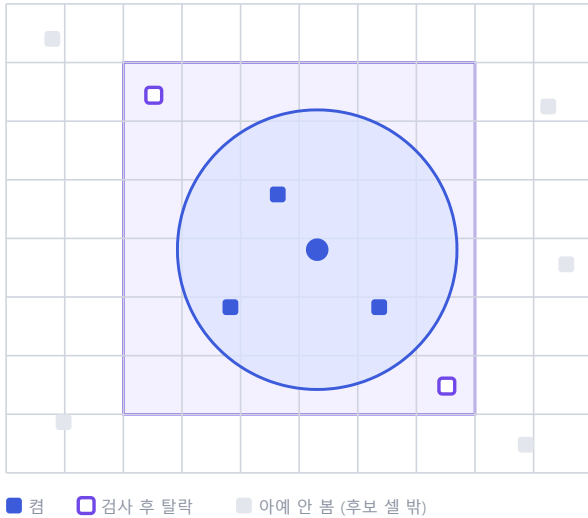
등록 대상은 몬스터가 아니라 스포너

스포너는 움직이지 않으니 위치를 한 번만 등록하면 된다. 몬스터를 등록하면 매 프레임 위치를 다시 써야 하고, 개수도 10~20배다. 몬스터 무리를 "스포너 하나"로 대표시키는 것 자체가 이 장르에 맞는 선택이다. (움직이는 몬스터 문제는 5절 ⑧)

4. 방식 C — 청크로 후보를 좁히고, 거리로 판정한다 (A + B)

충돌 판정의 Broad phase / Narrow phase와 같은 구조. 청크 = "누구를 검사할지", 거리 = "컬지 말지".

⑦ 두 단계 판정



1단계 Broad — 청크

반경의 외접 사각형이 걸치는 셀만 꺼낸다 (보라 영역). 이 셀들에 등록된 스폰너 k개만 후보.

2단계 Narrow — 거리

후보 k개만 실제 거리 + 히스테리시스로 판정. 판정 비용이 전체 N이 아니라 k에 비례.

덤 — 반경을 층마다 따로

청크는 로드(에셋)에, 거리는 존재(오브젝트)에 쓴다. 0절 ②의 층 구조가 이 조합에서 자연스럽게 나온다.

솔직한 단서

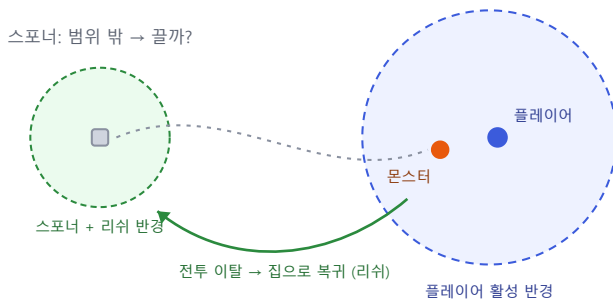
지금 맵은 스폰너 23개라 전수 거리 검사도 사실상 공짜다. 이 조합의 값어치는 CPU보다 맵이 커져도 구조가 안 바뀐다는 것과 격자를 로드·AI·판정에 재사용한다는 데 있다.

널리 알려진 예: 마인크래프트는 청크를 로드하는 거리(렌더 거리)와 개체를 실제로 돌리는 거리(시뮬레이션 거리)를 따로 둔다. 언리얼 World Partition은 월드를 격자 셀로 나누고, 플레이어 같은 '스트리밍 소스' 주변 반경의 셀만 로드한다.

5. 어느 방식을 골라도 남는 문제 — 움직이는 엔티티

스폰너는 가만히 있지만 몬스터는 플레이어를 쫓아 움직인다. Sector·청크·거리 전부 똑같이 부딪힌다.

⑧ 스폰너는 범위 밖, 몬스터는 플레이어 바로 옆



정적인 것(스폰너·NPC) = 공간으로 판정
청크/거리/무엇이든.

움직이는 것(몬스터) = 자기 상태로 판정

전투 중이면 스폰너가 꺼져도 살아 있다 → 전투가 끝나면 리쉬로 집 복귀 → 도착 후 스폰너 상태를 따른다. 유예 상한(N초)을 뒤서 영원히 안 꺼지는 경우를 막는다.

결론

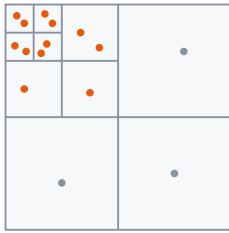
이건 방식 선택이 아니라 소유권 규칙으로 푼다. 1절 패치 ②(전투 중 유지)와 리쉬가 여기서 다시 쓰인다.

6. 그 밖의 방식들

D는 A의 변형, E·F·G·H는 다른 층을 다루는 방식이다 — A·B·C와 경쟁하지 않고 겹쳐 쓴다.

D 계층 공간 분할 (쿼드트리)

존재 층 · A의 변형



백백한 곳만 더 잘게 쪼갬다. 격자의 "셀 크기 하나로는 못 맞춘다" 문제에 대한 답.

- 밀도가 극단적으로 불균일한 맵에 강함
- 움직이는 개체는 매번 트리 재배치 비용
- 고도차 없는 평면 맵이라 옥트리까지는 불필요

언제: 한쪽에 몬스터 수백, 나머지는 텅 빈 맵. 우리 맵(무리 단위·고른 배치)엔 균일 격자로 충분할 가능성이 높다.

E 중요도 · LOD 텍 등급

텍(품질) 층 · 끄지 않고 느리게



아무도 사라지지 않는다. 먼 것은 **생각(BT 평가)**만 드물게, 이동·회전 보간은 매 프레임.

- **밀집에서 효과가 있는 유일한 방식** — 다 가까이 있어도 등급을 나눈다
- 팝인이 원천적으로 없다
- 메모리는 1바이트도 안 준다 / 총량 N에 여전히 비례
- 전제: 몬스터마다 도는 **Update()** 를 매니저 하나로 모아 야 함

언제: A·B·C 위에 겹쳐서. 카오스 던전형(밀도 문제)에서는 주력.

F 엔티티 가상화 — 레코드와 오브젝트 분리

레코드 층 · 화면 밖 세계를 멈추지 않는 법

레코드 — 항상 있음

point[0] 살아있음
point[1] 죽음
재등장 = 12:04:31
point[2] 비어 있음

절대 시각으로 기록 → 꺼져 있는 동안에도 시간은 흐른 것으로 친다 (어빌리티 쿨다운과 같은 방식)

구체화

환원

오브젝트 — 근처만

GameObject · NavMesh
BT · 애니메이터 · HP바

풀에서 꺼낼 때 레코드를 읽고
풀에 넣을 때 레코드에 쓴다

"몬스터가 존재한다"와 "몬스터 오브젝트가 있다"를 분리한다. 오브젝트는 레코드의 **보이는 얼굴**일 뿐.

- 꺼졌다 커져도 리스폰 타이머·처치 상태가 이어진다
- 오브젝트를 풀까지 반납해도 세계는 유지 → **메모리를 실제로 줄일 수 있는 전제**
- 지금은 이 레코드를 스포너가 **BaseMonsterController[]** 로 쥐고 있어 오브젝트와 한 몸이다

언제: A·B·C 어떤 걸 쓰든 같이 필요. 반오픈월드의 "한 번 잡은 적은 그대로" 같은 영속 상태의 기반.

G 지역 스트리밍

로드 층 · 오픈월드 / 반오픈월드



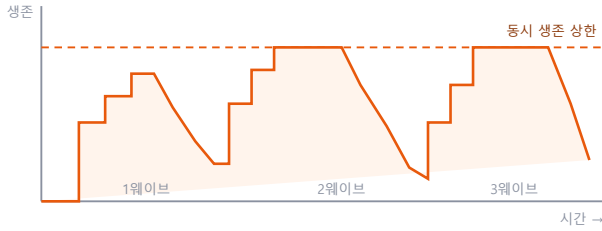
존재 판정보다 한 단계 바깥 — **에셋·지형을 메모리에 올릴지**를 정한다.

- 반오픈월드: 맵을 몇 개로 끊고 전환 시 로딩(텔레포트·문). 맵 하나 안에서는 다시 A~C
- 오픈월드: 격자 셀을 반경 기준으로 끊김 없이 로드 — C를 로드 층에 쓴 것
- 이 프로젝트의 Addressable 리소스 스코프가 바로 이 층의 부품

언제: 맵 하나를 통째로 올리면 메모리가 모자랄 때. 현재 BackStreetMap 규모엔 불필요.

H 인카운터 · 웨이브 예산

존재 층 · 카오스 던전형 (공간이 아니라 시간으로)



아레나가 좁아 전부 화면 안 → A~C는 **거를 게 없다**. 대신 수와 속도를 제어한다.

- 동시 생존 상한 + 프레임당 스폰 수 제한(스파이크 분산)
- 풀 선(先)워밍 — 웨이브 최대치만큼 미리
- 개체당 비용 줄이기: 개별 경로탐색 대신 공유 경로(플로우 필드), 매니저 일괄 업데이트, 인스턴싱/ECS

언제: 밀도가 문제일 때. E와 한 세트. 이 프로젝트에 이 장면을 넣을지는 별도 결정.

참고: 서버의 관심 영역 (AOI)

이 프로젝트는 완전 싱글 — 범위 밖, 정리만 해 둔다

MMO의 필드 몬스터는 서버가 시뮬레이션하고, 서버가 "이 클라에게 무엇을 보낼지"를 격자·거리로 정한다(Interest Management). 그 경우 **존재 층은 서버로 올라가고**, 클라에는 받은 것을 얼마나 싸게 보여줄지(틱·연출 품질)만 남는다.

- 그래서 A~C를 클라에서 직접 만들어 보는 것 = 서버 AOI를 로컬에서 재현해 보는 것과 같은 공부
- 반대로 E(품질 층)는 서버가 있어도 클라 몫으로 남는다

7. 한 장 비교

"중다/나쁘다"가 아니라 "무엇을 풀고 무엇을 남기나". 층이 다른 방식끼리는 우열 비교가 아니라 조합 대상이다.

방식	층	저작 비용	경계 문제	판정 비용	맵이 커지면	밀집	메모리
현재 Sector	존재	높음 (수동)	있음	트리거 이벤트 (≈0)	구역도 커짐	무력	안 줄 (풀 유지)
A 체크	존재·로드	0 (자동)	이웃 셀 + 유지 범위로 해소	O(1) 셀 계산	비용 고정	무력	셀 = 로드 단위 가능
B 거리	존재	0	없음 (히스테리시스)	O(N) 전수 — CullingGroup으로 저렴	N에 비례	무력	컬 것만
C 체크+거리	존재·로드	0	없음	O(k) 후보만	비용 고정	무력	로드/존재 반경 분리
D 쿼트리	존재	0	없음	O(log N) · 재배치 비용	중음	밀도 편차엔 강함	—
E 중요도/LOD	틱	0	해당 없음 (고지 않음)	버킷 관리	N에 비례	유일하게 효과	안 줄
F 가상화	레코드	레코드 설계	—	—	필수에 가까움	—	줄일 수 있는 전제
G 지역 스트리밍	로드	중간 (레벨 분할)	로딩 경계	—	이게 답	—	실제로 줄
H 웨이브 예산	존재	웨이브 데이터	없음 (아레나)	—	해당 없음	상한·예산으로 제어	선위밍 필요

표를 읽는 법

A·B·C·D는 같은 칸(존재 층)을 두고 경쟁한다 — 여기서 하나를 고른다. E·F·G·H는 다른 칸이다 — 고른 것 위에 필요한 만큼 얹는다. "체크 vs 거리 vs 둘 다"라는 원래 질문의 답은 C에 가깝고, 진짜 질문은 "그 위에 무엇을 얹을까"다.

8. 장면별 조합

0절의 세 장면에 층별로 무엇을 끼우는가.

⑨ 장면 × 층

	연속 필드 (← 우리 맵)	좁은 아레나 대량	오픈·반오픈월드
① 틱	E 중요도/LOD 실측 보고 필요할 때만	E 중요도/LOD + 일괄 업데이트 여기가 주 전장	E 중요도/LOD 면 개체 · 군중
② 존재	C 체크 + 거리 + 움직이는 몬스터 소유권 규칙	H 웨이브 · 동시 상한 공간 분할은 무의미	C 체크 + 거리 로드된 지역 안에서
③ 로드	맵 통째 로드 현재 규모에선 스트리밍 불필요	던전 통째 로드 + 풀 선위밍 입장 로딩에 비용을 몰아둔다	G 지역 스트리밍 여기가 주 전장
④ 레코드	F 스폰서 소유 레코드 리스폰 시각 = 절대 시각	거의 불필요 일회성 인스턴스	F 영속 레코드 처치·채집 상태 저장

진한 칸 = 그 장면의 주 전장. 장면마다 주 전장이 다른 층에 있다 — 0절에서 "세 문제는 다른 문제"라고 한 이유다.

9. 이 프로젝트에 대한 제안

출발점(Sector) → 다음 한 걸음. 결정은 아직 아니고, 다음 대화에서 정할 것들을 모아 둔다.

제안 — 존재 층을 C(청크 + 거리)로

- ① Sector의 네 문제(경계·단위·노가다·스파이크)를 **한 번에 없앤다** — 패치가 아니라 판정 기준 교체.
- ② 청크 격자가 **로드 층(리소스 시스템)**과 **판정 쿼리(어빌리티 광역기)**에 재사용될 자리를 만든다 — 세 대주제를 잇는 고리.
- ③ 맵을 키우거나 두 번째 필드를 붙여도 구조가 그대로다.
단, 우리 규모(스포너 23개)에선 B(거리만)로도 충분히 돌아간다. **B로 먼저 만들고, 청크는 "후보 추리기"로 나중에 끼우는 순서도 가능하다.**

다음에 정해야 할 것

1. **범위** — 세 장면 중 무엇을 이 프로젝트에서 실제로 만들까? (연속 필드는 확정. 좁은 아레나 대량(H+E)을 추가할지 / 오픈월드(G)는 설명만 할지)
2. **반경** — 화면 $\approx 25m$ 기준으로 진입·이탈 반경 (후보: 35m / 50m). 셀 크기 S (후보: 화면 한 번 정도 = 25m 전후)
3. **등록 단위** — 스폰너로 확정해도 되는지 (NPC·채집물도 같은 등록기에 넣을지)
4. **움직이는 몬스터 규칙** — 전투 중 유예, 리쉬 복귀, 유예 상한 N초
5. **레코드 분리(F)** — 스폰너가 쥔 `BaseMonsterController[]` 를 포인트별 상태 레코드로 바꿀지, 리스폰 타이머를 절대 시각으로 옮길지
6. **Sector의 운명** — 삭제 / 게임플레이 구역(지역 이름·BGM·필드 보스) 용도로 역할 변경
7. **측정** — 무엇으로 "나아졌다"를 말할지. 기존 MonsterSimulation 트랙의 계측(활성 수·히칭·팝인)을 그대로 가져올지

선행 조건 (기존 점검에서 이미 나온 것)

몬스터 생명주기 계약(OnSpawn/OnDespawn 미배선)과 풀 프리워밍 부족(풀 30 vs 섹터 최대 119)은 **어떤 방식을 고르든** 먼저 성해야 한다. 방식을 바꾸면 스폰/디스폰 횟수가 늘어나는 쪽이라, 이 둘이 깨져 있으면 방식의 차이가 아니라 풀·생명주기 결함을 재게 된다. (근거: `docs/MonsterSimulation/Audit/Lifecycle.md`, `Comparison/07_Prerequisites.md` — 현재 미커밋 작업에서 진행 중일 수 있음)